Federated Learning Optimization

A Communication-Aware and Fairness-Aware Federated Learning Pipeline on UCI HAR

Author Name Placeholder

April 26, 2026

Abstract

This report explains a federated learning optimization project implemented in Python and PyTorch. The project trains a human activity recognition model using the UCI Human Activity Recognition Using Smartphones dataset. Each human subject is treated as one federated client. Instead of gathering all client data into one central training table, the project simulates federated learning: a global model is sent to selected clients, each client trains locally, and the server averages the resulting model weights. The project studies four optimization ideas: FedAvg convergence, communication cost, CVaR-style fairness for weak clients, and hyperparameter search using grid search and differential evolution. The final pipeline produces training summaries, fairness summaries, communication shadow-price results, search results, plots, and a draft report.

Contents

1 Executive Summary **3**
- 1.1 Problem Solved . . . 3
- 1.2 Dataset . . . 3
- 1.3 Model and ML Approach . . . 3
- 1.4 Final Outputs . . . 3
- 1.5 Main Results . . . 4

2 Repository Structure **4**
- 2.1 File Tree . . . 4
- 2.2 Folder and File Purpose . . . 5

3 End-to-End Pipeline Overview **5**
- 3.1 Workflow . . . 5
- 3.2 Stage-by-Stage Explanation . . . 6

4 Dataset and EDA **6**
- 4.1 Dataset Source . . . 6
- 4.2 Available Files . . . 6
- 4.3 Dataset Shape . . . 7
- 4.4 Label Distribution . . . 7
- 4.5 EDA Interpretation . . . 7
- 4.6 Recommended EDA Code . . . 7

5 Preprocessing **8**
- 5.1 Label Conversion . . . 8
- 5.2 Feature Scaling . . . 8

6 Model and Algorithms **8**
- 6.1 HARMLP Architecture . . . 8
- 6.2 FedAvg Objective . . . 8
- 6.3 FedAvg Aggregation . . . 9

7 Training **9**
- 7.1 Main Training Configuration . . . 9
- 7.2 Local Client Training . . . 9
- 7.3 Loss Function . . . 10

8 Evaluation Metrics **10**
- 8.1 Accuracy . . . 10
- 8.2 Average Client Accuracy . . . 10
- 8.3 Worst-Client Accuracy . . . 10

9 CVaR Fairness **10**
- 9.1 Idea . . . 10
- 9.2 Code . . . 10
- 9.3 CVaR Results . . . 11

10 Communication-Budget LP **11**
- 10.1 Communication Cost . . . 11
- 10.2 LP Formulation . . . 11
- 10.3 KKT Checks . . . 11

11 Hyperparameter Search **12**
11.1 Search Variables . . . 12
11.2 Fitness Function . . . 12
11.3 Grid Search Results . . . 12
11.4 Differential Evolution Result . . . 12

12 Results **12**
12.1 FedAvg Results . . . 12
12.2 Interpretation . . . 13

13 File-by-File Explanation **13**
13.1 `flopt/config.py` . . . 13
13.2 `flopt/data.py` . . . 13
13.3 `flopt/models.py` . . . 13
13.4 `flopt/fedavg.py` . . . 13
13.5 `flopt/duality.py` . . . 14
13.6 `flopt/search.py` . . . 14
13.7 `flopt/plots.py` . . . 14
13.8 `experiments/run_full.py` . . . 14

14 How Files Call Each Other **14**

15 Beginner Explanation **14**

16 Issues, Risks, and Limitations **15**

17 Suggested Improvements **15**

18 Reproducibility Instructions **15**
18.1 Install Dependencies . . . 15
18.2 Run Main Experiment . . . 15
18.3 Run With Differential Evolution . . . 15
18.4 Common Errors . . . 16

19 Appendix: Glossary **16**

20 Appendix: Missing Items **16**

1 Executive Summary

1.1 Problem Solved

The project asks a practical machine learning question: can a federated learning system be accurate, communication-efficient, and fair to weak clients at the same time?

Federated learning is useful when raw data should not be moved into one central server. In this project, each person in the UCI HAR dataset is treated as one client. The model travels to the clients, trains locally, and only model weights return to the server.

1.2 Dataset

The project uses the UCI Human Activity Recognition Using Smartphones dataset. It contains smartphone accelerometer and gyroscope features from 30 volunteers. The task is to predict one of six activities:

1. WALKING
2. WALKING_UPSTAIRS
3. WALKING_DOWNSTAIRS
4. SITTING
5. STANDING
6. LAYING

1.3 Model and ML Approach

The main model is a small multilayer perceptron called `HARMLP`. It maps 561 input features to 6 output activity classes:

$$561 \to 128 \to 64 \to 6.$$

The training algorithm is FedAvg, short for Federated Averaging. In every round, a subset of clients receives the global model, trains locally, and sends updated weights back. The server averages those weights to form the next global model.

1.4 Final Outputs

The cleaned final output folder is `outputs/full_uci`. It contains:

- `convergence_mean.png`
- `cvar_pareto.png`
- `shadow_price.png`
- `ga_vs_grid.png`
- `fedavg_summary.csv`
- `cvar_summary.csv`
- `grid_search.csv`
- `lp_shadow_price.csv`
- `lp_shadow_price.json`
- `ga_result.json`
- `report_draft.md`

1.5 Main Results

Metric	Value
Mean FedAvg accuracy	0.9339
Mean FedAvg loss	0.1700
Mean worst-client accuracy	0.6914
Best CVaR alpha by worst-client accuracy	0.9 or 0.95
Best CVaR worst-client accuracy	0.7037
Best grid search policy	$E = 4, K = 10, \eta = 0.02$
Best grid search fitness	0.9082
Differential evolution fitness	0.3993
Differential evolution evaluations	36
LP budgets with positive shadow price	5 out of 10

Table 1: Main results from the generated output files.

Precision, recall, F1 score, confusion matrix, saved model checkpoints, and example predictions are not available from the provided files. To report them, the project would need to save predictions, labels, and trained model weights.

2 Repository Structure

2.1 File Tree

```
federated-learning-optimization/
  README.md
  requirements.txt
  .gitignore

  flopt/
    __init__.py
    config.py
    data.py
    models.py
    fedavg.py
    duality.py
    search.py
    plots.py

  experiments/
    run_full.py

  data/
    UCI HAR Dataset/
      README.txt
      features.txt
      features_info.txt
      activity_labels.txt
      train/
        X_train.txt
        y_train.txt
        subject_train.txt
      test/
        X_test.txt
        y_test.txt
```

```
        subject_test.txt

  outputs/
    full_uci/
      convergence_mean.png
      cvar_pareto.png
      shadow_price.png
      ga_vs_grid.png
      fedavg_summary.csv
      cvar_summary.csv
      grid_search.csv
      lp_shadow_price.csv
      lp_shadow_price.json
      ga_result.json
      report_draft.md
```

2.2 Folder and File Purpose

Path	What it contains	Why it exists
`README.md`	Short project summary and run command.	Helps a new user understand the project quickly.
`requirements.txt`	Python dependencies.	Makes environment setup reproducible.
`.gitignore`	Ignore rules for generated folders.	Prevents committing virtual environments, data, and outputs.
`flopt/config.py`	Federated learning configuration class.	Keeps training settings organized.
`flopt/data.py`	UCI HAR loader.	Converts raw files into federated clients.
`flopt/models.py`	Model definitions.	Defines the neural networks.
`flopt/fedavg.py`	FedAvg training and evaluation.	Core training engine.
`flopt/duality.py`	LP and KKT analysis.	Computes communication shadow prices.
`flopt/search.py`	Grid search and differential evolution.	Finds good training settings.
`flopt/plots.py`	Plot functions.	Creates report figures.
`experiments/run_full.py`	Main experiment script.	Runs the full pipeline.
`data/UCI HAR Dataset`	Dataset files.	Provides real activity data.
`outputs/full_uci`	Final outputs.	Stores metrics, plots, and report draft.

3 End-to-End Pipeline Overview

3.1 Workflow

```
Raw UCI HAR text files
    -> NumPy arrays
    -> Standardized features
    -> 30 ClientData objects
    -> HARMLP model
    -> FedAvg training
    -> Evaluation metrics
    -> CVaR fairness sweep
```

```
    -> Grid search and differential evolution
    -> Communication-budget LP
    -> Plots, CSV files, JSON files, and report draft
```

3.2 Stage-by-Stage Explanation

Stage	What happens	Code file
Initialization	The script parses command-line arguments and creates the output folder.	`experiments/run_full.py`
Data loading	UCI HAR text files are read into arrays.	`flopt/data.py`
Preprocessing	Features are standardized using training-set statistics.	`flopt/data.py`
Client creation	Each subject becomes one federated client.	`flopt/data.py`
Model definition	The HAR MLP is created.	`flopt/models.py`
Training	FedAvg trains the global model.	`flopt/fedavg.py`
Evaluation	Average loss, average accuracy, and worst-client accuracy are computed.	`flopt/fedavg.py`
Fairness	CVaR-style client weighting is tested.	`flopt/fedavg.py`
Search	Grid search and differential evolution search over E, K, η.	`flopt/search.py`
Duality	LP solves communication-budget policy selection.	`flopt/duality.py`
Results	CSV, JSON, PNG, and report files are written.	`run_full.py`, `plots.py`

4 Dataset and EDA

4.1 Dataset Source

The UCI HAR dataset contains sensor recordings from 30 volunteers aged 19 to 48. Each volunteer wore a smartphone on the waist. Accelerometer and gyroscope signals were sampled at 50 Hz. The original dataset provides 561 engineered features from time and frequency domains.

4.2 Available Files

- `train/X_train.txt`: training features.
- `train/y_train.txt`: training labels.
- `train/subject_train.txt`: subject IDs for train rows.
- `test/X_test.txt`: test features.
- `test/y_test.txt`: test labels.
- `test/subject_test.txt`: subject IDs for test rows.
- `activity_labels.txt`: class name mapping.

4.3 Dataset Shape

Property	Value
Training rows	7352
Test rows	2947
Feature columns	561
Activity classes	6
Train subjects	21
Test subjects	9
Total subjects	30
Missing values in train features	0
Missing values in test features	0
Duplicate train feature rows	0
Feature minimum before project scaling	-1.0
Feature maximum before project scaling	1.0

Table 4: UCI HAR dataset facts computed from the local files.

4.4 Label Distribution

Class ID	Activity	Train Count	Test Count
1	WALKING	1226	496
2	WALKING_UPSTAIRS	1073	471
3	WALKING_DOWNSTAIRS	986	420
4	SITTING	1286	491
5	STANDING	1374	532
6	LAYING	1407	537

Table 5: Activity class distribution.

4.5 EDA Interpretation

The classes are not perfectly balanced, but there is no extreme imbalance. The features have no missing values in the loaded files. The dataset already contains engineered numeric features, so no text tokenization, image processing, or embedding generation is needed.

4.6 Recommended EDA Code

```python
import numpy as np
from pathlib import Path

root=Path("data/UCI HAR Dataset")
x_train=np.loadtxt(root/"train/X_train.txt")
y_train=np.loadtxt(root/"train/y_train.txt",dtype=int)
subjects=np.loadtxt(root/"train/subject_train.txt",dtype=int)

print("Shape:",x_train.shape)
print("Missing values:",np.isnan(x_train).sum())
print("Feature min:",x_train.min())
print("Feature max:",x_train.max())

labels,counts=np.unique(y_train,return_counts=True)
```

```
for label,count in zip(labels,counts):
    print(label,count)

subject_ids,subject_counts=np.unique(subjects,return_counts=True)
print("Number of subjects:",len(subject_ids))
print("Rows per subject:",dict(zip(subject_ids,subject_counts)))
```

This code reads the train data, checks shape, checks missing values, shows the feature range, counts labels, and counts how many rows each subject contributes.

5 Preprocessing

5.1 Label Conversion

The UCI labels are numbered 1 through 6. PyTorch cross-entropy expects class labels from 0 through 5. Therefore, the code subtracts 1:

```
y_train=np.loadtxt(root/"train"/"y_train.txt",dtype="int64")-1
y_test=np.loadtxt(root/"test"/"y_test.txt",dtype="int64")-1
```

5.2 Feature Scaling

```
scaler=StandardScaler().fit(x_train)
x_train=scaler.transform(x_train).astype("float32")
x_test=scaler.transform(x_test).astype("float32")
```

The scaler is fitted only on training features. This avoids using test statistics to transform train data. The transformed arrays are converted to `float32`, which is the common tensor type for PyTorch neural networks.

6 Model and Algorithms

6.1 HARMLP Architecture

```
class HARMLP(nn.Module):
    def __init__(self,features:int=561,classes:int=6):
        super().__init__()
        self.net=nn.Sequential(
            nn.Linear(features,128),
            nn.ReLU(),
            nn.Linear(128,64),
            nn.ReLU(),
            nn.Linear(64,classes),
        )

    def forward(self,x:torch.Tensor)->torch.Tensor:
        return self.net(x)
```

The model receives one 561-dimensional feature vector and outputs six raw scores called logits. The predicted class is the index of the largest logit.

6.2 FedAvg Objective

The global federated objective is:

$$F(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w),$$

where K is the number of clients, n_k is the number of examples on client k, n is the total number of examples, and $F_k(w)$ is the loss on client k.

6.3 FedAvg Aggregation

After local training, the server averages selected client weights:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_k.$$

7 Training

7.1 Main Training Configuration

Setting	Value
Dataset	UCI HAR
Clients	30
Rounds	30
Seeds	7, 11, 19
Default local epochs	2
Default clients per round	10
Default learning rate	0.03
Batch size	32
Optimizer	SGD
Loss	Cross entropy

Table 6: Main experiment configuration.

7.2 Local Client Training

```python
def train_one_client(model,client,cfg,device):
    model.train()
    x=torch.tensor(client.x_train,dtype=torch.float32)
    y=torch.tensor(client.y_train,dtype=torch.long)
    loader=DataLoader(TensorDataset(x,y),batch_size=cfg.batch_size,shuffle=True)
    opt=torch.optim.SGD(model.parameters(),lr=cfg.lr)
    loss_fn=nn.CrossEntropyLoss()
    for _ in range(cfg.local_epochs):
        for xb,yb in loader:
            xb=xb.to(device)
            yb=yb.to(device)
            opt.zero_grad()
            loss=loss_fn(model(xb),yb)
            loss.backward()
            opt.step()
```

This is ordinary neural network training, but it happens separately on each selected client. The model makes predictions, the loss measures mistakes, backpropagation computes gradients, and SGD updates weights.

7.3 Loss Function

For one example with logits z and true label y, cross-entropy is:

$$\mathcal{L}(z, y) = -\log \frac{\exp(z_y)}{\sum_{c=1}^{C} \exp(z_c)}.$$

High loss means the model is wrong or uncertain. Low loss means the model gives a high score to the correct class.

8 Evaluation Metrics

8.1 Accuracy

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{number of predictions}}.$$

8.2 Average Client Accuracy

$$\text{MeanAccuracy} = \frac{1}{K} \sum_{k=1}^{K} \text{Accuracy}_k.$$

This treats each client equally.

8.3 Worst-Client Accuracy

$$\text{WorstClientAccuracy} = \min_k \text{Accuracy}_k.$$

This metric is important because high average accuracy can hide one or two badly performing clients.

9 CVaR Fairness

9.1 Idea

CVaR-style weighting gives extra importance to high-loss clients. The code computes a quantile threshold:

$$\tau = \text{Quantile}_\alpha(\ell_1, \ell_2, \ldots, \ell_K).$$

Then it computes the tail amount:

$$\text{tail}_k = \max(\ell_k - \tau, 0).$$

Clients with larger tail values receive more aggregation weight.

9.2 Code

```python
def _aggregation_weights(sizes,losses,cfg):
    weights=sizes.astype("float64")/sizes.sum()
    if cfg.cvar_alpha<=0:
        return weights
    tau=np.quantile(losses,cfg.cvar_alpha)
    tail=np.maximum(losses-tau,0)
    if tail.sum()==0:
        return weights
    weights=weights*(1+cfg.fairness_strength*tail/tail.sum())
    return weights/weights.sum()
```

9.3 CVaR Results

Alpha	Accuracy	Accuracy Std	Worst Accuracy	Worst Accuracy Std
0.00	0.9352	0.0033	0.6996	0.0354
0.50	0.9351	0.0039	0.6955	0.0407
0.75	0.9356	0.0038	0.6996	0.0354
0.90	0.9366	0.0035	0.7037	0.0363
0.95	0.9366	0.0035	0.7037	0.0363

Table 7: CVaR sweep results.

Image not found in this Overleaf project. Upload the corresponding generated PNG image to display this figure.

Figure 1: Average client accuracy versus worst-client accuracy for CVaR settings.

10 Communication-Budget LP

10.1 Communication Cost

The code estimates communication with:

$$\text{upload bytes} = \text{parameters} \times 4 \times \text{selected clients},$$

$$\text{download bytes} = \text{parameters} \times 4 \times \text{selected clients}.$$

The factor 4 means each parameter is assumed to be a 32-bit float.

10.2 LP Formulation

The LP is over candidate training policies, not over neural network weights. If policy i has loss ℓ_i and communication cost c_i, then:

$$\begin{aligned} \min_x \quad & \sum_i \ell_i x_i \\ \text{s.t.} \quad & \sum_i x_i = 1, \\ & \sum_i c_i x_i \le B, \\ & x_i \ge 0. \end{aligned}$$

The dual variable $\lambda^\star$ attached to the budget constraint is interpreted as the shadow price of communication.

10.3 KKT Checks

The code checks primal feasibility, dual feasibility, complementary slackness, and a stationarity residual. Five out of ten budget levels had positive shadow price in the generated results.

Image not found in this Overleaf project. Upload the corresponding generated PNG image to display this figure.

Figure 2: Communication budget versus LP dual variable.

11 Hyperparameter Search

11.1 Search Variables

The project searches over:

- E: local epochs,
- K: clients per round,
- η: learning rate.

11.2 Fitness Function

$$\text{fitness} = \text{loss} + \gamma \cdot \text{communication} + 10 \max(0, 0.80 - \text{accuracy}).$$

Lower fitness is better.

11.3 Grid Search Results

E	K	LR	Accuracy	Loss	Fitness
4	10	0.02	0.9045	0.2636	0.9082
2	10	0.03	0.8836	0.3333	0.9779
4	15	0.02	0.9041	0.2694	1.2363
1	10	0.03	0.7749	0.6535	1.5493
2	15	0.01	0.7613	0.8605	2.2148
1	5	0.01	0.6546	1.1620	2.9388

Table 8: Grid search results.

11.4 Differential Evolution Result

The differential evolution output was:

$$x = [4.8939, 4.1545, 0.0717],$$

with fitness 0.3993 after 36 evaluations. This is approximately $E = 5$, $K = 4$, and learning rate 0.0717.

Image not found in this Overleaf project. Upload the corresponding generated PNG image to display this figure.

Figure 3: Best grid-search fitness versus differential evolution fitness.

12 Results

12.1 FedAvg Results

Seed	Accuracy	Loss	Total Communication	Worst-Client Accuracy
7	0.9272	0.1819	193396800	0.6296
11	0.9355	0.1674	193396800	0.7037
19	0.9391	0.1606	193396800	0.7407
Mean	0.9339	0.1700	193396800	0.6914

Table 9: FedAvg summary over three seeds.

Image not found in this Overleaf project. Upload the corresponding generated PNG image to display this figure.

Figure 4: Mean FedAvg convergence across seeds.

12.2 Interpretation

The model performs well on average, with approximately 93.4 percent mean client accuracy. The worst-client accuracy is lower, approximately 69.1 percent, which shows that average accuracy alone is not enough. CVaR-style weighting slightly improves the weakest-client result. The LP analysis shows that communication can be treated as a constrained resource with a measurable shadow price.

13 File-by-File Explanation

13.1 `flopt/config.py`

This file defines `FLConfig`, a small immutable configuration object.

```
@dataclass(frozen=True)
class FLConfig:
    rounds:int=20
    local_epochs:int=2
    clients_per_round:int=10
    lr:float=0.01
    batch_size:int=32
    seed:int=7
    cvar_alpha:float=0.0
    fairness_strength:float=1.0
```

If this file is removed, training scripts lose the central object that stores rounds, learning rate, batch size, seed, and fairness settings.

13.2 `flopt/data.py`

This file loads UCI HAR data and creates client datasets. Its most important class is:

```
@dataclass
class ClientData:
    x_train:np.ndarray
    y_train:np.ndarray
    x_test:np.ndarray
    y_test:np.ndarray
```

Each `ClientData` object represents one subject. If this file is removed, the project cannot load data or build clients.

13.3 `flopt/models.py`

This file defines `HARMLP`, `LogisticModel`, `count_parameters`, and `clone_model`. The main executed model is `HARMLP`. If this file is removed, no trainable model exists.

13.4 `flopt/fedavg.py`

This is the training engine. It contains local training, global aggregation, evaluation, and CVaR-style weighting. If this file is removed, the project cannot train.

13.5 flopt/duality.py

This file solves the communication-budget LP using CVXPY. It also computes KKT diagnostics. If this file is removed, the shadow-price contribution disappears.

13.6 flopt/search.py

This file implements grid search and differential evolution. If this file is removed, the project cannot automatically compare training configurations.

13.7 flopt/plots.py

This file creates plots and configures Matplotlib to use local cache folders. If it is removed, the project can still train, but report-ready figures will not be created.

13.8 experiments/run_full.py

This is the main entry point. It calls the loader, trainer, fairness sweep, search code, LP code, plot code, and report writer. If it is removed, the modules still exist, but there is no single command to run the complete project.

14 How Files Call Each Other

```
experiments/run_full.py
  -> flopt/config.py
  -> flopt/data.py
  -> flopt/models.py
  -> flopt/fedavg.py
  -> flopt/search.py
  -> flopt/duality.py
  -> flopt/plots.py

flopt/search.py
  -> flopt/fedavg.py
  -> flopt/models.py

flopt/fedavg.py
  -> flopt/config.py
  -> flopt/data.py
  -> flopt/models.py
```

15 Beginner Explanation

Imagine 30 people each have phone sensor data. We want one model that recognizes activities, but we do not want to put everyone's data in one giant table. So the server sends a model to some people. Each person trains the model on their own data. Then they send back only the updated model weights. The server averages those weights. After many rounds, the shared model becomes better.

The model sees 561 numbers per example. Those numbers describe motion patterns. The model tries to learn which motion pattern means walking, sitting, standing, and so on.

The fairness part asks whether the model is helping the weakest clients. The communication part asks how expensive it is to send model weights. The search part asks which training settings give the best tradeoff.

16 Issues, Risks, and Limitations

- Only three seeds were used.
- Raw per-round JSON logs were removed during cleanup.
- No saved model checkpoint is available.
- No precision, recall, F1 score, confusion matrix, or example predictions are available from the provided files.
- Differential evolution used a short evaluation budget.
- CVaR weighting is a practical heuristic, not a formal proof of CVaR convergence.
- Communication cost is estimated from parameter count, not measured network traffic.

17 Suggested Improvements

- Add a dedicated EDA notebook.
- Save raw per-round logs for auditability.
- Save trained model checkpoints.
- Add confusion matrix, precision, recall, and F1 score.
- Add per-client metric tables.
- Run more seeds.
- Validate the best GA configuration using the full 30 or 100 rounds.
- Compare against FedProx or SCAFFOLD.

18 Reproducibility Instructions

18.1 Install Dependencies

```
python -m venv .venv
.venv/bin/pip install -r requirements.txt
```

18.2 Run Main Experiment

```
.venv/bin/python experiments/run_full.py --source uci --rounds 30 --seeds 7,11,19
    --out outputs/full_uci --skip-ga
```

18.3 Run With Differential Evolution

```
.venv/bin/python experiments/run_full.py --source uci --rounds 30 --seeds 7,11,19
    --out outputs/full_uci
```

18.4 Common Errors

Error	Fix
Dataset download fails	Manually place the UCI HAR dataset under `data/UCI HAR Dataset`.
Module not found	Run from the project root and install dependencies in `.venv`.
Matplotlib cache error	The project sets local cache paths in `plots.py`; delete cache folders and rerun if needed.
CVXPY solver error	Ensure CVXPY and its solvers are installed from `requirements.txt`.
Long runtime	Use fewer rounds or pass `--skip-ga`.

19 Appendix: Glossary

Term	Meaning
Federated learning	Training where data stays on clients and only model updates move.
Client	A local data holder. Here, one subject is one client.
Server	The coordinator that averages client model updates.
FedAvg	Federated Averaging, the main algorithm.
Round	One cycle of client selection, local training, and aggregation.
Local epoch	One pass over a client's local training data.
Loss	A number measuring how wrong the model is.
Cross entropy	Standard loss for multiclass classification.
Accuracy	Fraction of correct predictions.
Worst-client accuracy	The accuracy of the weakest client.
CVaR	A risk idea used here to emphasize high-loss clients.
LP	Linear program.
Dual variable	A number showing how important a constraint is.
Shadow price	The marginal value of relaxing a constraint.
KKT conditions	Mathematical checks for constrained optimization.
Grid search	Trying a fixed list of hyperparameters.
Differential evolution	A global search algorithm.

20 Appendix: Missing Items

The following are not available from the provided files:

- Saved final model weights.
- Example predictions.
- Confusion matrix.
- Precision, recall, and F1 score.
- Full retained per-round JSON logs after cleanup.
- Formal convergence theorem.